TH5 — Control Unit, Microcode ROM and CPU Integration

CS2300 · Group G06

Chaithanya PS (CS25B018)
Yash Tulshyan (CS25B095)

1. Interface table

Every new subcircuit in th5.circ and its ports. tara_datapath, tara_decoder and IMM keep the interfaces given in the TH4 report; their only change is that the manual panel inputs are now driven by tara_cpu. Blocks from TH1–TH3 are unchanged.

Subcircuit	Port	Width	Notes
tstate_counter	CLK (in)	1	Common clock
	RST (in)	1	Clears the T-state counter to 0
	STEP_RESET (in)	1	From the control word; returns the counter to T0 on the next clock
	TSTATE_HALT (in)	1	From the control word (HALT); freezes the counter
	TSTATE (out)	4	Current T-state, T0–T15
ROM_ADDRESS	Z (in)	1	Zero flag
	N (in)	1	Negative flag
	OPCODE (in)	5	IR[15:11] from the decoder
	TSTATE_OUT (in)	4	Current T-state
	ROM_ADDR (out)	11	`{Z, N, OPCODE, TSTATE}`: Z is bit 10, N bit 9, OPCODE bits 8:4, T-state bits 3:0
control_rom	ROM_ADDR (in)	11	2048-word address
	CONTROL (out)	25	Control word (Section 2.1)
tara_control	TSTATE_OUT, Z, N, OPCODE (in)	4, 1, 1, 5	Passed to ROM_ADDRESS
	STEP_RST, MAR_LOAD, MDR_LOAD, IR_LOAD, ACC_LOAD, REG_WE, MEM_READ, MEM_WRITE, STATUS_WE, IMM, PC_LOAD, PC_FETCH_EN, HALT, SPARE_1, SPARE_2 (out)	1 each	The 25-bit control word split into named fields (Section 2.1)
	IMM_BIT_EXTEND (out)	2	Control-word bits 11:10
	ALU_OPCODE (out)	5	Control-word bits 16:12
	BUS_SOURCE (out)	3	Control-word bits 19:17
tara_cpu	CLK (in)	1	Drives every clocked block
	RST (in)	1	Reset
	TSTATE (out)	4	Current T-state

Subcircuit	Port	Width	Notes
	PC_COUNT (out)	16	Program counter value
	RAM_WORD (out)	16	ram_word output
	MAR_OUT, MDR_OUT, IR_OUT (out)	16 each	Memory address, memory data and instruction registers
	R0 – R7 (out)	16 each	Register-file contents (probes for the trace sheet)

Wiring inside tara_cpu, at pin level: the decoder reads IR_OUT; OPCODE goes to tara_control and its fields go to the datapath register-index and immediate inputs; the control-word outputs drive the datapath pins of the same name, except that ALU_OPCODE drives the datapath `OPCODE` pin, STEP_RST drives `STEP_RESET` and HALT drives `TSTATE_HALT` on the counter. The IMM bit overrides IMM8 with 0xFF through a constant and an 8-bit multiplexer.

2. Design decisions

2.1 Control word

The control word is 25 bits wide. Bit 0 is the least significant bit. Every field is a direct output of tara_control, so a ROM word is the complete state of the datapath controls for one clock.

Field	LSB	Width	Function
STEP_RST	0	1	Return the T-state counter to T0 after this clock
MAR_LOAD	1	1	Load MAR from the bus
MDR_LOAD	2	1	Load MDR from the bus
IR_LOAD	3	1	Load IR from the bus
ACC_LOAD	4	1	Load ACC from the ALU result
REG_WE	5	1	Write the bus into the destination register
MEM_READ	6	1	Read ram_word at MAR
MEM_WRITE	7	1	Write MDR to ram_word at MAR
STATUS_WE	8	1	Capture Z, N, C, V
IMM	9	1	Force IMM8 to 0xFF (used by LIH)
IMM_BIT_EXTEND	10	2	00 zero-extend imm8; 01 sign-extend imm8; 10 imm8 into bits 15:8; 11 zero-extend imm5
ALU_OPCODE	12	5	ALU operation; equal to the instruction opcode for ALU instructions
BUS_SOURCE	17	3	000 PC, 001 MDR, 010 IR, 011 IMM, 100 register file port A, 101 ACC, 110 ram_word, 111 bus off
PC_LOAD	20	1	Clock the program counter
PC_FETCH_EN	21	1	0: PC ← PC + 2; 1: PC ← bus
HALT	22	1	Freeze the T-state counter
SPARE_1	23	1	Unused
SPARE_2	24	1	Unused

Idle word: `SAFE` = 0x00E0001 (bus off, nothing loaded or written, STEP_RST = 1). Every ROM address not used by an implemented instruction holds `SAFE`.

2.2 ROM addressing and T-states

The ROM has 2048 words of 25 bits. The address is `{Z, N, OPCODE, TSTATE}`, so each opcode owns 16 consecutive rows, and each of the four Z/N combinations holds its own copy. Execution runs through T-states T0, T1, T2 and so on. The counter advances by one on every clock, returns to T0 when STEP_RST is asserted, and stops when HALT is asserted. An instruction therefore takes (number of executed states + 3) clocks.

Z and N are part of the address so that conditional branches (TH6) can select a different word without extra logic. No TH5 instruction uses them, so the four copies of every row are identical, which the generator checks.

2.3 Fetch sequence (T0–T2)

Fetch is the same for every opcode and every Z/N value. It is stored at T0–T2 of all 32 opcodes because IR still holds the previous instruction while these states run.

State	Word	Fields asserted	Micro-operation
T0	0000002	BUS_SOURCE=000, MAR_LOAD	MAR ← PC
T1	01c0044	BUS_SOURCE=110, MEM_READ, MDR_LOAD, PC_LOAD, PC_FETCH_EN=0	MDR ← Mem[MAR]; PC ← PC + 2
T2	0020008	BUS_SOURCE=001, IR_LOAD	IR ← MDR

In TH4 the read, the PC advance and the MDR load were set together and MDR took the word one pulse later. In TH5 this is resolved by clocking `ram_word` from the inverted clock, so that it latches mid-way through T1 and MDR captures valid data at the following rising edge (Fault 2, Section 5).

2.4 Microcode for the implemented instructions

Execution starts at T3. Addresses and words are for the Z = N = 0 rows. `A` is the ALU input fed from the register file and `B` is the bus value, as wired in tara_datapath.

Instr	Op	State	Addr	Word	Fields asserted	Micro-operation
NOP	0	T3	0x003	00e0001	STEP_RST	idle
HLT	1	T3	0x013	04e0000	HALT	halt the counter
MOV	2	T3	0x023	0069c10	ACC_LOAD, IMM_BIT_EXTEND=3, ALU_OPCODE=9, BUS_SOURCE=3	ACC ← Rs + 0 (imm5 = 0)
		T4	0x024	00a0021	STEP_RST, REG_WE, BUS_SOURCE=5	Rd ← ACC
LIL	3	T3	0x033	0060021	STEP_RST, REG_WE, BUS_SOURCE=3	Rd ← zext(imm8)
LIH	4	T3	0x043	006d210	ACC_LOAD, IMM, ALU_OPCODE=13, BUS_SOURCE=3	ACC ← Rd AND 0x00FF
		T4	0x044	00a0020	REG_WE, BUS_SOURCE=5	Rd ← ACC
		T5	0x045	006e810	ACC_LOAD, IMM_BIT_EXTEND=2, ALU_OPCODE=14, BUS_SOURCE=3	ACC ← (imm8 << 8) OR Rd
		T6	0x046	00a0021	STEP_RST, REG_WE, BUS_SOURCE=5	Rd ← ACC
ADD	9	T3	0x093	0089010	ACC_LOAD, ALU_OPCODE=9, BUS_SOURCE=4	ACC ← Ra + Rb
		T4	0x094	00a0021	STEP_RST, REG_WE, BUS_SOURCE=5	Rd ← ACC
SUB	10	T3	0x0A3	008a010	ACC_LOAD, ALU_OPCODE=10, BUS_SOURCE=4	ACC ← Ra − Rb
		T4	0x0A4	00a0021	STEP_RST, REG_WE, BUS_SOURCE=5	Rd ← ACC

Instr	Op	State	Addr	Word	Fields asserted	Micro-operation
ADDI	11	T3	0x0B3	0069410	ACC_LOAD, IMM_BIT_EXTEND=1, ALU_OPCODE=9, BUS_SOURCE=3	ACC ← Rd + sext(imm8)
		T4	0x0B4	00a0021	STEP_RST, REG_WE, BUS_SOURCE=5	Rd ← ACC
AND	13	T3	0x0D3	008d010	ACC_LOAD, ALU_OPCODE=13, BUS_SOURCE=4	ACC ← Ra AND Rb
		T4	0x0D4	00a0021	STEP_RST, REG_WE, BUS_SOURCE=5	Rd ← ACC
OR	14	T3	0x0E3	008e010	ACC_LOAD, ALU_OPCODE=14, BUS_SOURCE=4	ACC ← Ra OR Rb
		T4	0x0E4	00a0021	STEP_RST, REG_WE, BUS_SOURCE=5	Rd ← ACC
XOR	15	T3	0x0F3	008f010	ACC_LOAD, ALU_OPCODE=15, BUS_SOURCE=4	ACC ← Ra XOR Rb
		T4	0x0F4	00a0021	STEP_RST, REG_WE, BUS_SOURCE=5	Rd ← ACC
NOT	16	T3	0x103	0090010	ACC_LOAD, ALU_OPCODE=16, BUS_SOURCE=4	ACC ← NOT Rs
		T4	0x104	00a0021	STEP_RST, REG_WE, BUS_SOURCE=5	Rd ← ACC
SHL	17	T3	0x113	0071c10	ACC_LOAD, IMM_BIT_EXTEND=3, ALU_OPCODE=17, BUS_SOURCE=3	ACC ← Rd << imm5
		T4	0x114	00a0021	STEP_RST, REG_WE, BUS_SOURCE=5	Rd ← ACC
SHR	18	T3	0x123	0072c10	ACC_LOAD, IMM_BIT_EXTEND=3, ALU_OPCODE=18, BUS_SOURCE=3	ACC ← Rd >> imm5
		T4	0x124	00a0021	STEP_RST, REG_WE, BUS_SOURCE=5	Rd ← ACC
SLT	19	T3	0x133	0093010	ACC_LOAD, ALU_OPCODE=19, BUS_SOURCE=4	ACC ← (Ra < Rb)
		T4	0x134	00a0021	STEP_RST, REG_WE, BUS_SOURCE=5	Rd ← ACC

Cycle counts follow from the table: NOP, LIL and HLT-to-halt take 4 clocks; MOV, ADD, SUB, ADDI, AND, OR, XOR, NOT, SHL, SHR and SLT take 5; LIH takes 7. HLT never asserts STEP_RST, so the counter stays at T3.

2.5 ROM generator and consistency checks

G06_th5_microcode_gen.py builds the ROM from named fields (a mistyped field name or an oversized value raises an error), then runs nine checks and writes the Logisim hex file only if all pass. All nine pass for the submitted ROM.

- One bus source per word, and no load from an idle bus.
- Every implemented instruction reaches STEP_RST or HALT.
- MEM_READ and MEM_WRITE are never asserted together.
- Write enables are never asserted without valid source data.
- Non-conditional opcodes give identical words for all four Z/N values.
- Undefined opcodes modify no architectural state.
- Fetch (T0–T2) is present for every opcode and condition.
- PC_LOAD is asserted only at T1, so the PC advances once per instruction.
- Every word fits in 25 bits.

3. Test evidence

Probe values are shown in hexadecimal. A clock pulse is a 0→1 transition of CLK, and the state after n clock pulses is the state after n rising edges from reset. Every test is run from the reset state, with the program loaded into ram_word at address 0. The expected values below were worked out from the microcode in Section 2.4 before running; the screenshots should be compared against them.

3.1 Unit tests of the control blocks

#	Block and stimulus	Expected
U1	tstate_counter: RST=1 for one clock, then RST=0 and 5 clocks with STEP_RESET=0, HALT=0	TSTATE: 0, 1, 2, 3, 4, 5 after the successive clocks
U2	tstate_counter: at TSTATE=4 set STEP_RESET=1 and clock once	TSTATE = 0
U3	tstate_counter: at TSTATE=3 set HALT=1 and clock three times	TSTATE stays 3
U4	tara_control: OPCODE=9, TSTATE_OUT=3, Z=N=0	ACC_LOAD=1, ALU_OPCODE=9, BUS_SOURCE=100, all other outputs 0
U5	tara_control: OPCODE=9, TSTATE_OUT=4	REG_WE=1, STEP_RST=1, BUS_SOURCE=101, all other outputs 0
U6	tara_control: OPCODE=1 (HLT), TSTATE_OUT=4	STEP_RST=1, BUS_SOURCE=111 (the idle word), all other outputs 0

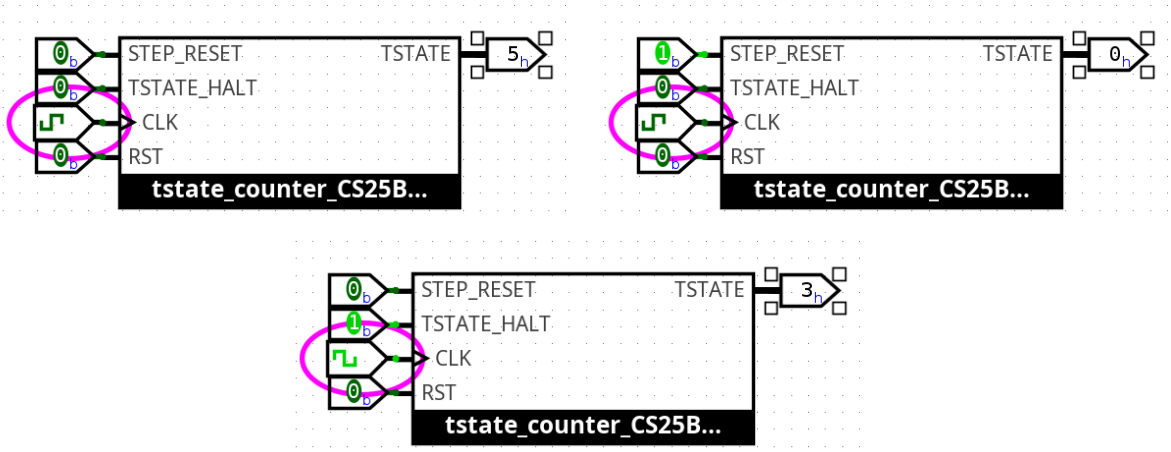


Figure 1 — tstate_counter tests: (a) U1, TSTATE = 5 after five clocks; (b) U2, STEP_RESET = 1 returns TSTATE to 0; (c) U3, HALT = 1 holds TSTATE at 3.

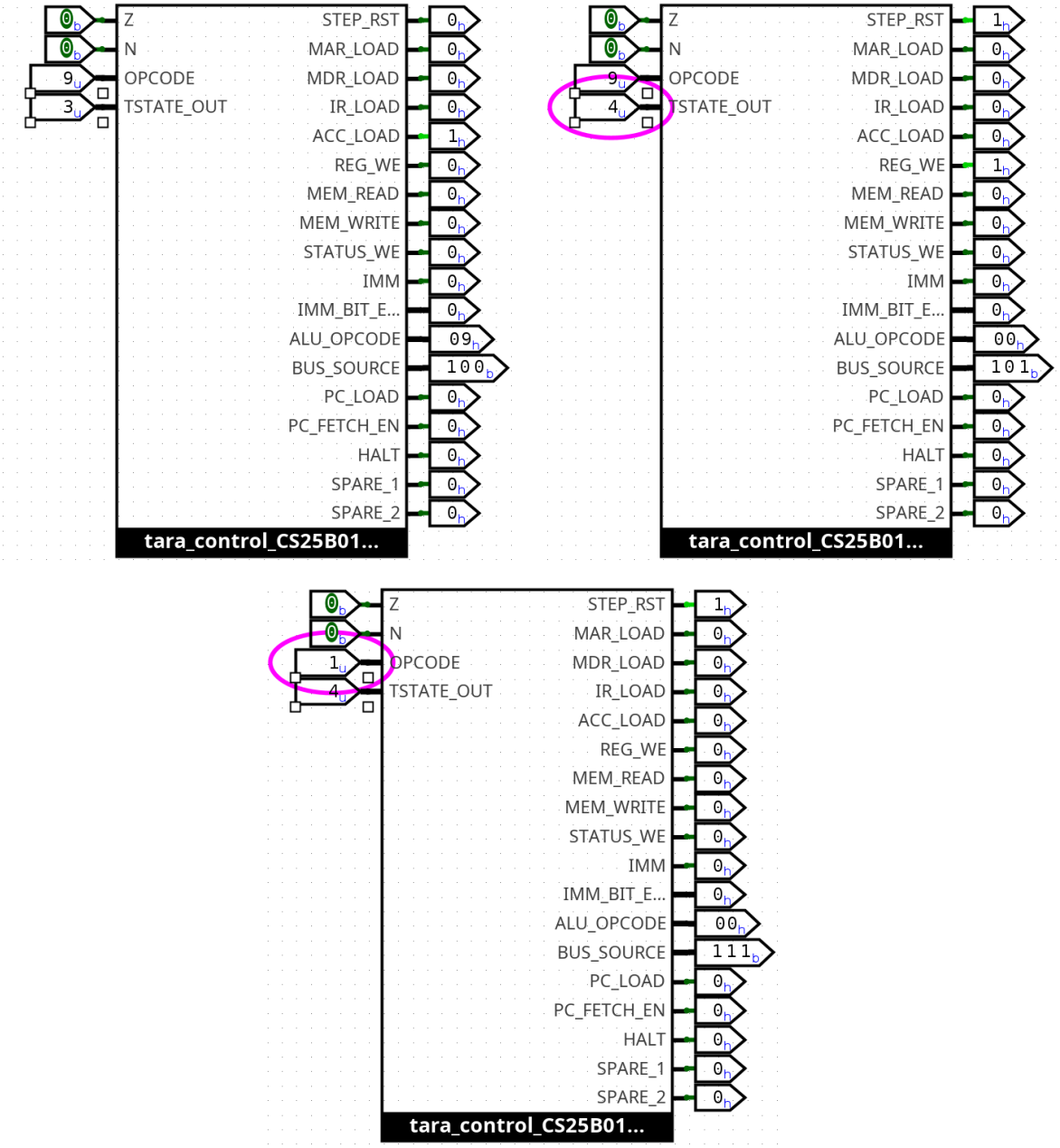


Figure 2 — tara_control tests: (a) U4, OPCODE = 9 at T3; (b) U5, OPCODE = 9 at T4; (c) U6, OPCODE = 1 (HLT) at T4.

3.2 Test program A

Eleven instructions followed by NOP and HLT, stored at addresses 0, 2, 4, … Fields are `OPCODE[15:11] RD[10:8] RA[7:5] RB[4:2]`, or `RD` and `imm8[7:0]` for immediate formats.

Address	Instruction	Encoding
0x00	LIL R0, 0x36	0x1836
0x02	LIL R1, 0x3E	0x193E
0x04	ADD R3, R0, R1	0x4B04
0x06	SUB R4, R1, R0	0x5420
0x08	LIH R0, 0x12	0x2012
0x0A	ADDI R1, −1	0x59FF
0x0C	MOV R5, R3	0x1560
0x0E	NOT R6, R0	0x8600

Address	Instruction	Encoding
0x10	SHL R4, 3	0x8C03
0x12	SHR R3, 2	0x9302
0x14	SLT R2, R1, R0	0x9A20
0x16	NOP	0x0000
0x18	HLT	0x0800

3.3 Full-program run (program A)

Clock counts are cumulative from reset. After each instruction TSTATE is back at 0 and PC_COUNT points at the next instruction. IR_OUT holds the instruction just executed.

#	Instruction	Clk	PC	IR	R0	R1	R2	R3	R4	R5	R6	R7
TC1	LIL R0, 0x36	4	0x02	0x1836	0036	0000	0000	0000	0000	0000	0000	0000
TC2	LIL R1, 0x3E	8	0x04	0x193E	0036	003E	0000	0000	0000	0000	0000	0000
TC3	ADD R3, R0, R1	13	0x06	0x4B04	0036	003E	0000	0074	0000	0000	0000	0000
TC4	SUB R4, R1, R0	18	0x08	0x5420	0036	003E	0000	0074	0008	0000	0000	0000
TC5	LIH R0, 0x12	25	0x0A	0x2012	1236	003E	0000	0074	0008	0000	0000	0000
TC6	ADDI R1, −1	30	0x0C	0x59FF	1236	003D	0000	0074	0008	0000	0000	0000
TC7	MOV R5, R3	35	0x0E	0x1560	1236	003D	0000	0074	0008	0074	0000	0000
TC8	NOT R6, R0	40	0x10	0x8600	1236	003D	0000	0074	0008	0074	EDC9	0000
TC9	SHL R4, 3	45	0x12	0x8C03	1236	003D	0000	0074	0040	0074	EDC9	0000
TC10	SHR R3, 2	50	0x14	0x9302	1236	003D	0000	001D	0040	0074	EDC9	0000
TC11	SLT R2, R1, R0	55	0x16	0x9A20	1236	003D	0001	001D	0040	0074	EDC9	0000
TC12	NOP	59	0x18	0x0000	1236	003D	0001	001D	0040	0074	EDC9	0000

Notes on the table: TC5 is the longest instruction (seven clocks) and R0 changes only at its last state. TC11 assumes SLT returns 0x0001 when true; 0x003D < 0x1236 holds for both signed and unsigned comparison. TC6 sign-extends 0xFF to 0xFFFF, so the addition subtracts one.

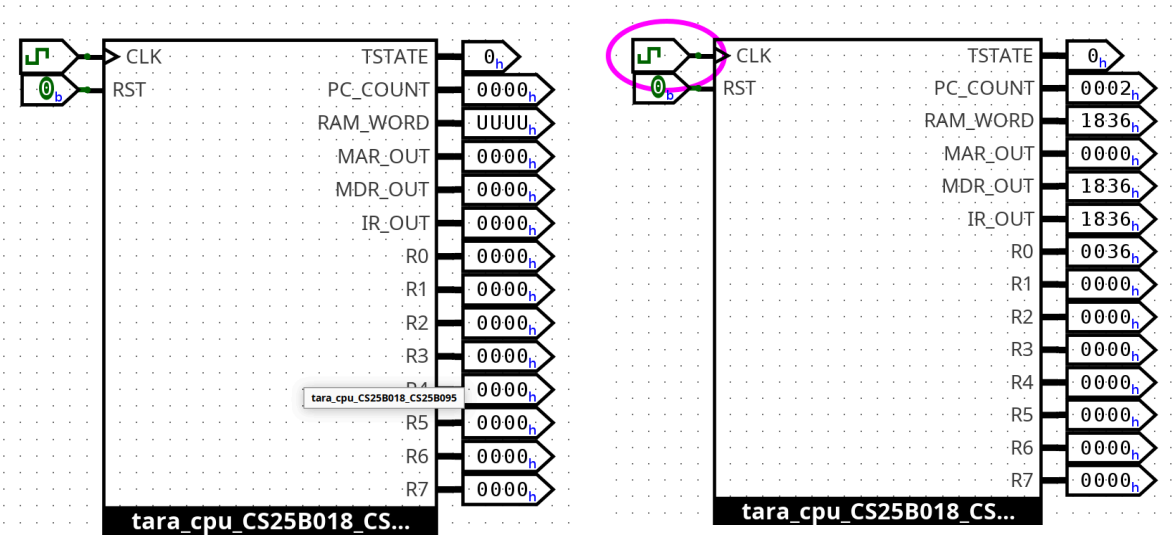


Figure 3 — tara_cpu running program A: (a) after reset, all probes 0; (b) after 4 clocks, LIL R0, 0x36 complete (R0 = 0x0036, PC_COUNT = 0x0002).

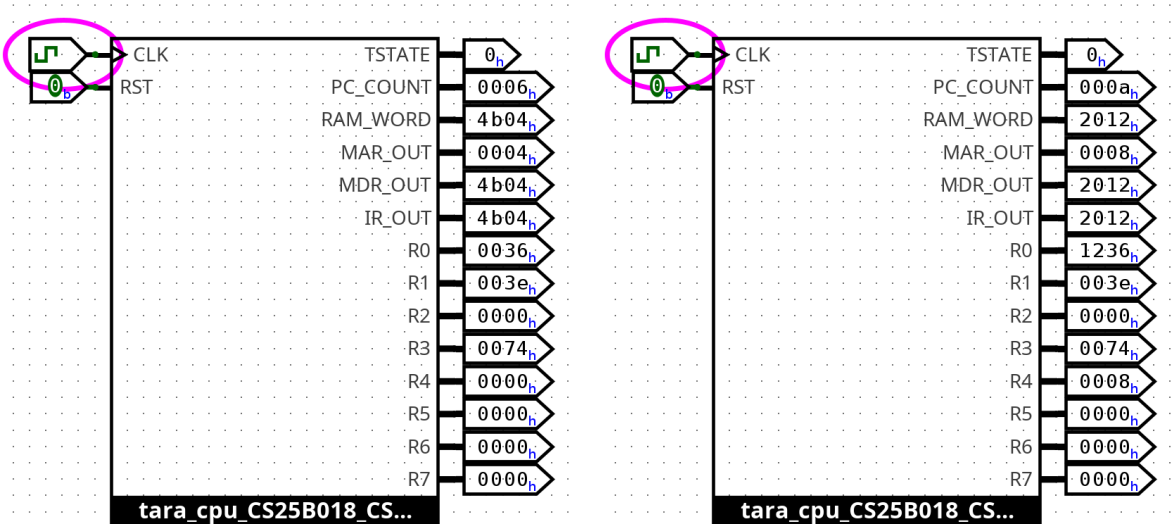


Figure 4 — (a) after 13 clocks, ADD R3, R0, R1 complete (R3 = 0x0074); (b) after 25 clocks, LIH R0, 0x12 complete (R0 = 0x1236).

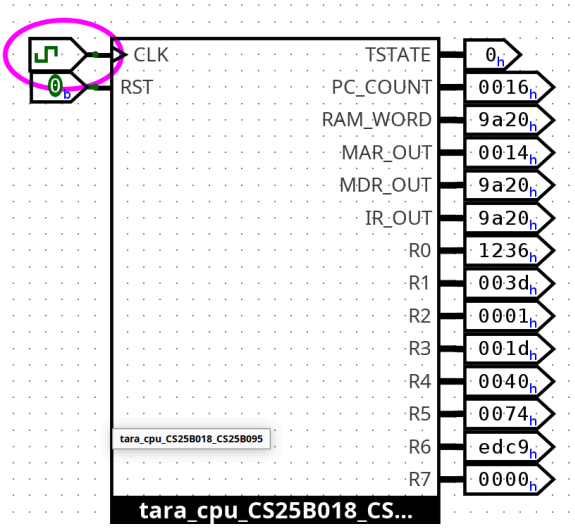


Figure 5 — after 55 clocks: end of the register-changing instructions (R0–R7 as in the last register row of the table).

3.4 Fetch and execute, state by state

Single-instruction walk-through using ADD R3, R0, R1 (TC3, clocks 9–13, PC = 4 at the start). Take one screenshot after each clock.

Clk	TSTATE after	Expected on the probes	Control word at the state just executed
9	1	MAR_OUT = 0x0004	T0: MAR_LOAD, BUS_SOURCE=000
10	2	RAM_WORD = 0x4B04; MDR_OUT = 0x4B04; PC_COUNT = 0x0006	T1: MEM_READ, MDR_LOAD, PC_LOAD, BUS_SOURCE=110
11	3	IR_OUT = 0x4B04	T2: IR_LOAD, BUS_SOURCE=001
12	4	No visible register change (ACC holds 0x0074)	T3: ACC_LOAD, ALU_OPCODE=9, BUS_SOURCE=100
13	0	R3 = 0x0074; PC_COUNT = 0x0006	T4: REG_WE, STEP_RST, BUS_SOURCE=101

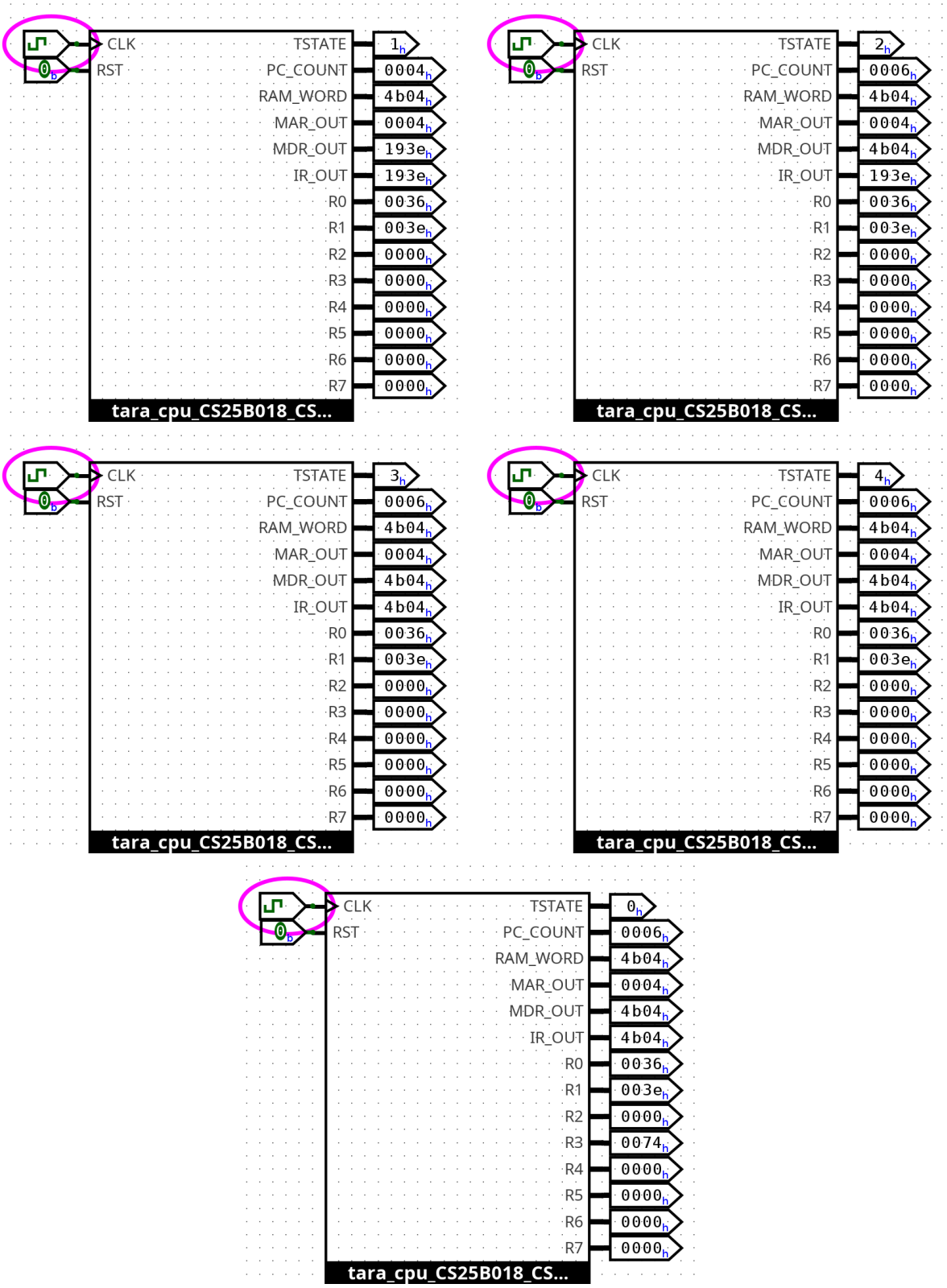


Figure 6 — ADD R3, R0, R1 stepped one clock at a time: (a) clock 9, MAR_OUT = 0x0004; (b) clock 10, PC_COUNT = 0x0006 and MDR_OUT = 0x4B04; (c) clock 11, IR_OUT = 0x4B04; (d) clock 12, execute state done (TSTATE = 4); (e) clock 13, R3 = 0x0074 and TSTATE = 0.

3.5 HLT, NOP and undefined opcodes

#	Stimulus	Expected	Screenshot
TC13	Program A: continue after TC12 (59 clocks) for 3 more clocks	TSTATE = 3; PC_COUNT = 0x001A; IR_OUT = 0x0800; registers unchanged	After clock 62

#	Stimulus	Expected	Screenshot
TC14	Continue for 10 more clocks	TSTATE stays 3; no probe changes	After clock 72
TC15	Pulse RST while halted	TSTATE returns to 0	Before and after

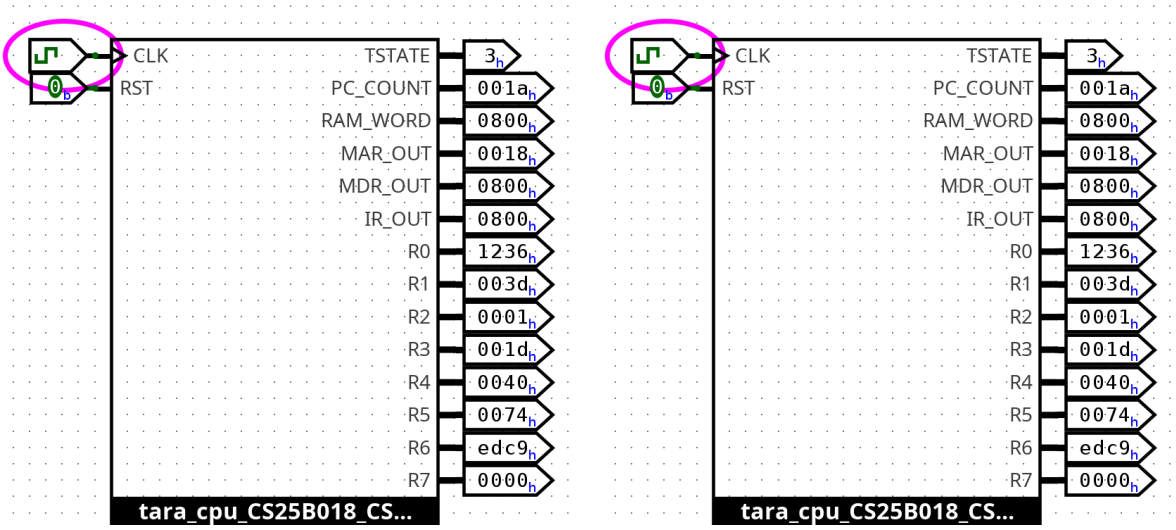


Figure 7 — HLT freezes the T-state counter: (a) TC13, TSTATE = 3 and PC_COUNT = 0x001A; (b) TC14, further clocks change nothing.

3.6 Test program B: AND, OR, XOR and an undefined opcode

Address	Instruction	Encoding
0x00	LIL R0, 0xF0	0x18F0
0x02	LIL R1, 0xCC	0x19CC
0x04	AND R2, R0, R1	0x6A04
0x06	OR R3, R0, R1	0x7304
0x08	XOR R4, R0, R1	0x7C04
0x0A	Undefined (opcode 31)	0xF800
0x0C	HLT	0x0800

#	Point in the run	Expected	Screenshot
TC16	After 8 clocks	R0 = 0x00F0; R1 = 0x00CC	Registers
TC17	After 13 clocks	R2 = 0x00C0 (0xF0 AND 0xCC)	R2
TC18	After 18 clocks	R3 = 0x00FC (0xF0 OR 0xCC)	R3
TC19	After 23 clocks	R4 = 0x003C (0xF0 XOR 0xCC)	R4
TC20	After 27 clocks (undefined opcode)	TSTATE = 0; PC_COUNT = 0x000C; IR_OUT = 0xF800; R0–R4 unchanged. It uses 4 clocks and changes nothing but IR and PC	All probes
TC21	After 30 clocks	HLT fetched; TSTATE = 3; PC_COUNT = 0x000E	Halted

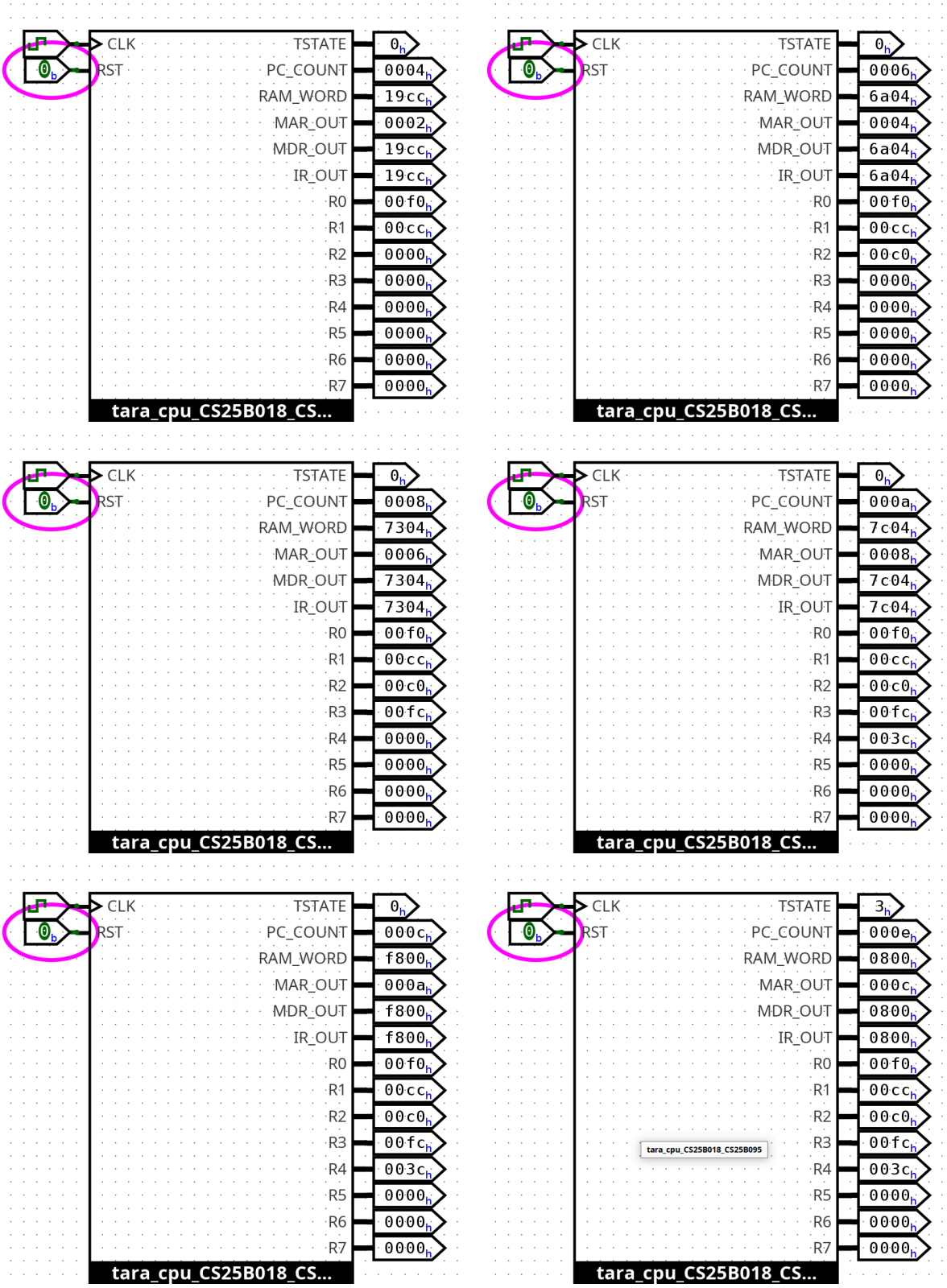


Figure 8 — program B: (a) TC16, R0 = 0x00F0 and R1 = 0x00CC; (b) TC17, R2 = 0x00C0 (AND); (c) TC18, R3 = 0x00FC (OR); (d) TC19, R4 = 0x003C (XOR); (e) TC20, undefined opcode 0xF800 changes only IR and PC; (f) TC21, HLT fetched, TSTATE = 3.

3.7 ROM generator output

Output of `python3 G06_th5_microcode_gen.py`: all nine checks print PASS and the file G06_th5_control_rom.hex is written with 2048 words × 25 bits and 15 instructions implemented.

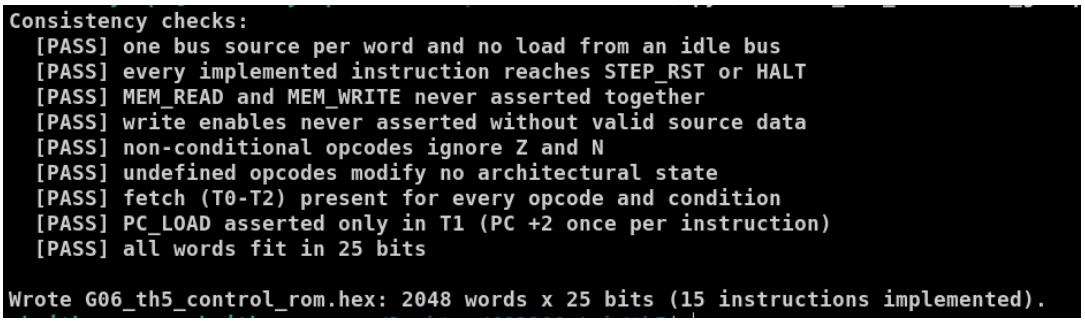


Figure 9 — generator output: all nine checks pass and the ROM file is written.

4. Control-signal inventory

Every control-word output, the pin it drives, and its value in the idle word. With every load and write bit at 0 no register changes state.

Signal	Width	Drives	Function	Idle
STEP_RST	1	tstate_counter STEP_RESET	Return to T0 on the next clock	1
MAR_LOAD	1	datapath MAR_LOAD	Load MAR from the bus	0
MDR_LOAD	1	datapath MDR_LOAD	Load MDR from the bus	0
IR_LOAD	1	datapath IR_LOAD	Load IR from the bus	0
ACC_LOAD	1	datapath ACC_LOAD	Capture the ALU result	0
REG_WE	1	datapath REG_WE	Write the bus to the destination register	0
MEM_READ	1	datapath MEM_READ	Read ram_word at MAR	0
MEM_WRITE	1	datapath MEM_WRITE	Write MDR to ram_word at MAR	0
STATUS_WE	1	datapath STATUS_WE	Capture Z, N, C, V	0
IMM	1	IMM8 override in tara_cpu	Force IMM8 to 0xFF	0
IMM_BIT_EXTEND	2	datapath IMM_BIT_EXTEND	Extension mode	00
ALU_OPCODE	5	datapath OPCODE	ALU operation	0
BUS_SOURCE	3	datapath BUS_SOURCE	Bus driver select; 111 = off	111
PC_LOAD	1	datapath PC_LOAD	Clock the PC	0
PC_FETCH_EN	1	datapath PC_FETCH_EN	0: PC + 2; 1: load from bus (not used in TH5)	0
HALT	1	tstate_counter TSTATE_HALT	Freeze the counter	0
SPARE_1, SPARE_2	1 each	not connected	Reserved	0

Unused in TH5 and left for TH6: PC_FETCH_EN = 1, MEM_WRITE, and Z/N-dependent rows.

5. Faults encountered

Fault 1 — SUB returned FFF8 instead of 0008

Cause. In our datapath the ALU's A input is the register-file output `QB` and B is the bus (`QA`). With the decoder's `RA` wired to `REG_A` and `RB` to `REG_B`, SUB computed `RB − RA`. ADD, AND and OR hid the problem because they are commutative.

Fix. The two register-index paths were swapped: `RA` now feeds `REG_B` (ALU A) and `RB` feeds `REG_A` (bus). SUB then gave `R1 − R0 = 0008`, and the same swap also fixed SLT and NOT. MOV could no

longer put its source on the bus, so it was rewritten as `ACC ← rs + 0` through the ALU, which makes it a 5-cycle instruction.

Fault 2 — first instruction executed as a NOP; PC stopped at 0016 instead of 0014

Cause. `ram_word` latches its output at the clock edge. During T1 the bus still carried the old RAM output, so MDR loaded stale data. The first fetch loaded 0 and ran as a NOP, and every later instruction executed one fetch late.

Fix. The RAM clock was inverted with a NOT gate in `tara_datapath`. The RAM now latches on the falling edge in the middle of T1, so its output is valid when MDR captures it at the next rising edge. Fetch stayed three states long, and `PC_COUNT` ended at `0014`.